

Project 3

zkVM-guest API Standardization

Standards Recap

Visual key: ■ Accepted ◆ In Review

1 ACCEPTED

RISC-V Target

Standardizes the ISA: riscv64im_zicclsm-unknown-none-elf. All conforming zkVMs must use this 64-bit target.

3 ACCEPTED

Cryptographic Accelerators C API

Standard header zkvm_accelerators.h exposing EVM precompile-equivalent operations across vendors.

5 ACCEPTED

Memory Layout Restrictions

Each vendor owns their linker script. Source portability is preserved via ISA and IO and accelerators standards.

7 IN REVIEW

Instruction Address Misaligned

Misaligned jump target → abnormal termination. Type-1 verifiers reject; Type-2 require non-success exit code.

2 ACCEPTED

IO Interface

Portable C functions read_input() / write_output() for private inputs and public outputs. Write once, run on any zkVM.

4 ACCEPTED

Execution Termination Semantics

Successful executions may produce a proof; failed ones must not. All panics and assertions map to this contract.

6 IN REVIEW

Memory Safety Guard Regions

Null-pointer trap (first 4 kB) and stack-overflow guard (4 kB below stack). Both trigger abnormal termination.

8 IN REVIEW

Static Library & Linker Script

Vendor must ship an .a archive with _start, IO, and accelerators, plus GNU ld / LLD compatible linker script.

New Standard: Instruction Address Misaligned

Status: In Review · Clarifies exception handling for jump targets

The Problem

Without the C (compressed) extension, RISC-V requires instructions to be 4-byte aligned. When a jump targets a non-aligned address, different zkVMs handle the resulting exception differently — some silently continue or round the address to 4-byte boundary, potentially producing a valid proof of broken execution.

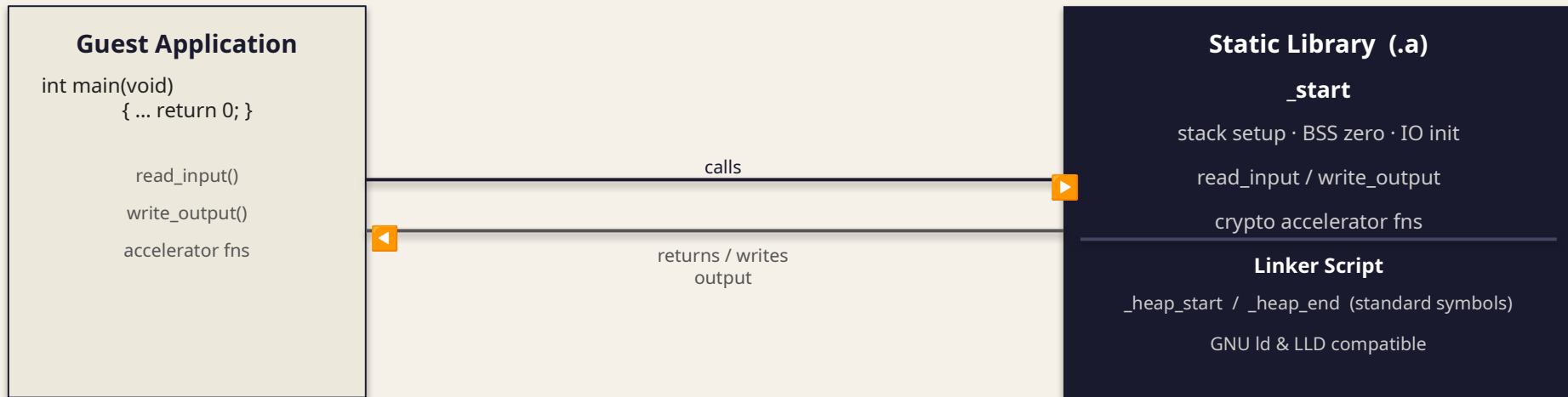
What It Mandates

- Instruction-address-misaligned exception → abnormal termination (per Termination Semantics standard)
- No recovery, no address rounding permitted
- Type 1 verifiers: must reject proofs of such executions
- Type 2 verifiers: must require a non-success exit code

Note: Zicclsm (data-misalignment support for loads/stores) is unrelated — this standard only covers instruction fetch.

New Standard: Static Library & Linker Script

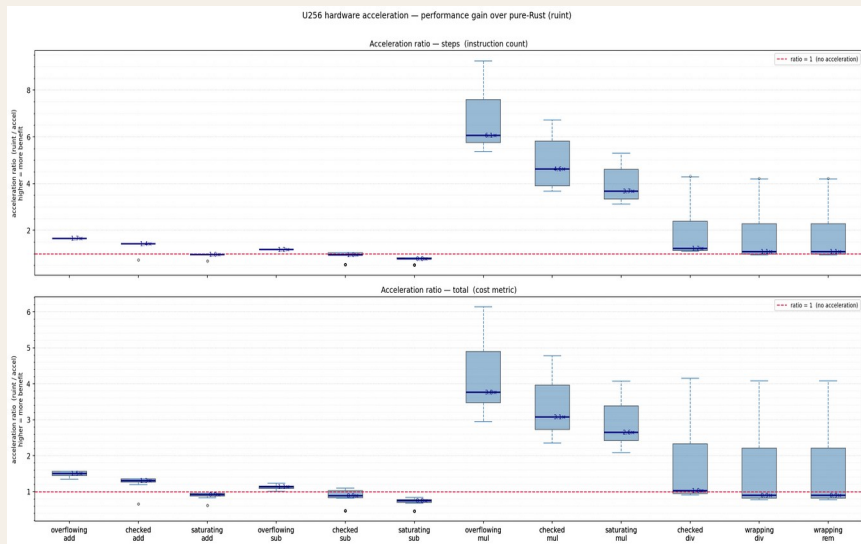
Status: In Review · Defines the vendor-supplied guest runtime



- `_start` must set up stack pointer, zero BSS, prepare IO, then call `main()`
- `main()` takes no arguments — zkVMs have no CLI; return 0 = success
- Linker script exposes `_heap_start` and `_heap_end` for custom allocators
- C++ support deferred to a future standard

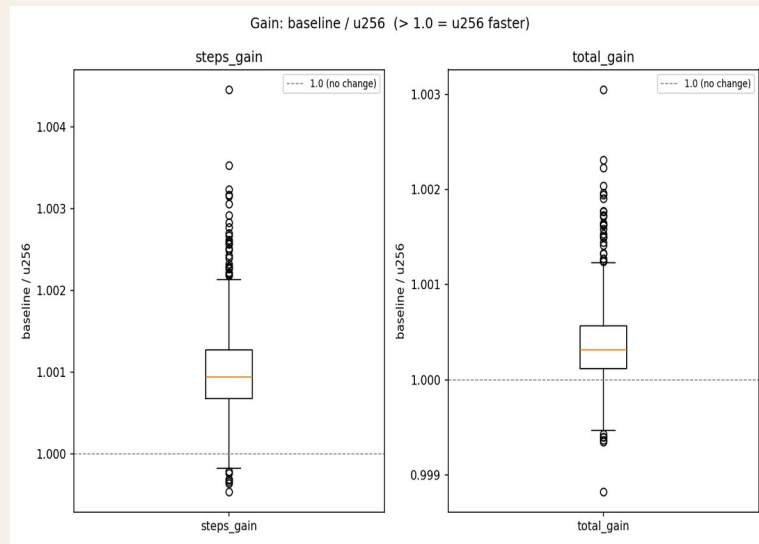
Exploring: U256 Acceleration Interface

Assessing whether standardizing a U256 C API is warranted for Ethereum block proving



Isolated operation acceleration ratio (one zkVM)

- mul variants: 2.6 – 6.1× isolated gain · survives end-to-end (median block +0.03%)
- add / overflowing_sub: modest isolated gain (1.01 – 1.56×) · end-to-end impact unclear
- saturating_sub / saturating_add: always slower than pure-Rust · must be excluded
- More investigation needed before proposing a standard — especially across diverse block workloads



End-to-end: 500 mainnet blocks (mul/div/rem accelerators)

Reminder: Standards Compliance

zkVM vendors are expected to comply with accepted standards.

64

Migrate to 64-bit RISC-V

The standardized target is RV64IM. Vendors currently running 32-bit zkVMs are expected to migrate.

Ø

Ship a no_std Rust SDK

The target riscv64im-unknown-none-elf is bare-metal — no OS, no std. SDKs that depend on std will not compile.

ZisK no_std SDK: github.com/0xPolygonHermes/zisk/pull/965

✓

Implement IO & Accelerator Interfaces

Guest programs must be able to call `read_input()/write_output()` and the accelerator API as specified.

SP1 compliance work: github.com/succinctlabs/sp1/pull/2763

The Ethereum Foundation can provide engineering support — reach out if you need help with standard implementation.